James Lewis - (S - 29)

Experiment 9 - Numpy Techniques

In [4]:
```
import numpy as np
```

In [5]:
```
# 1. Array Creation Techniques
print("1. Array Creation Techniques")
```

```
1. Array Creation Techniques
```

In [6]:
```
# a. Creating an array from a list
array_from_list = np.array([1, 2, 3, 4, 5])
print("Array from list:", array_from_list)
```

```
Array from list: [1 2 3 4 5]
```

In [7]:
```
# b. Using arange()
array_arange = np.arange(0, 10, 2)
print("Array using arange:", array_arange)
```

```
Array using arange: [0 2 4 6 8]
```

In [8]:
```
# c. Using linspace()
array_linspace = np.linspace(0, 10, 5)  # Divides 0 to 10 into 5 points
print("Array using linspace:", array_linspace)
```

```
Array using linspace: [ 0.   2.5  5.   7.5 10. ]
```

In [9]:
```
# d. Using zeros()
array_zeros = np.zeros((3, 3))
print("Array of zeros:\n", array_zeros)
```

```
Array of zeros:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

In [10]:
```
# e. Using ones()
array_ones = np.ones((2, 2))
print("Array of ones:\n", array_ones)
```

```
Array of ones:
 [[1. 1.]
 [1. 1.]]
```

In [11]:
```
# f. Using eye() for identity matrix
array_eye = np.eye(3)
print("Identity matrix:\n", array_eye)
```

```
Identity matrix:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [12]:
```python
# g. Using random() for random values
array_random = np.random.random((3, 3))
print("Random array:\n", array_random)
```

```
Random array:
 [[0.97705742 0.2639962  0.1398926 ]
 [0.61447848 0.45708154 0.07959132]
 [0.82007932 0.7173106  0.63894875]]
```

In [13]:
```python
# 2. Different NumPy Methods
print("\n2. NumPy Methods")
```

```
2. NumPy Methods
```

In [14]:
```python
# a. Reshaping an array
reshaped_array = np.arange(1, 10).reshape(3, 3)
print("Reshaped array:\n", reshaped_array)
```

```
Reshaped array:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [15]:
```python
# b. Transposing an array
transposed_array = reshaped_array.T
print("Transposed array:\n", transposed_array)
```

```
Transposed array:
 [[1 4 7]
 [2 5 8]
 [3 6 9]]
```

In [16]:
```python
# c. Mathematical operations
array_math = np.array([1, 2, 3])
print("Array + 2:", array_math + 2)
print("Array * 3:", array_math * 3)
print("Square root of array:", np.sqrt(array_math))
```

```
Array + 2: [3 4 5]
Array * 3: [3 6 9]
Square root of array: [1.         1.41421356 1.73205081]
```

In [17]:
```python
# d. Aggregation methods
print("Sum:", np.sum(array_math))
print("Mean:", np.mean(array_math))
print("Max:", np.max(array_math))
print("Min:", np.min(array_math))
```

```
Sum: 6
Mean: 2.0
Max: 3
Min: 1
```

In [18]:
```
# e. Concatenation of arrays
array_a = np.array([1, 2, 3])
array_b = np.array([4, 5, 6])
concat_array = np.concatenate((array_a, array_b))
print("Concatenated array:", concat_array)
```

```
Concatenated array: [1 2 3 4 5 6]
```

In [19]:
```
# f. Sorting an array
unsorted_array = np.array([3, 1, 4, 2])
sorted_array = np.sort(unsorted_array)
print("Sorted array:", sorted_array)
```

```
Sorted array: [1 2 3 4]
```

In [20]:
```
# g. Indexing and Slicing
indexed_value = array_math[1]  # Indexing
print("Indexed value:", indexed_value)
```

```
Indexed value: 2
```

In [21]:
```
sliced_array = array_math[1:3]  # Slicing
print("Sliced array:", sliced_array)
```

```
Sliced array: [2 3]
```

In [22]:
```
# h. Boolean Masking
boolean_mask = array_math > 2
print("Boolean mask:", boolean_mask)
print("Filtered values:", array_math[boolean_mask])
```

```
Boolean mask: [False False  True]
Filtered values: [3]
```